

PART A
(PART A : TO BE REFERRED BY STUDENTS)
Experiment No. 10(A)

Aim : To generate target code.

Objective: Understand intermediate code generation and code generation phase of compiler.
Develop a program to generate target code

Outcome: Students are able to implement a program to generate target code

Theory:

The last phase of compiler is the code generator. The intermediate representation of the source program is given as input to code generator and it produces target program as an output as shown in figure.

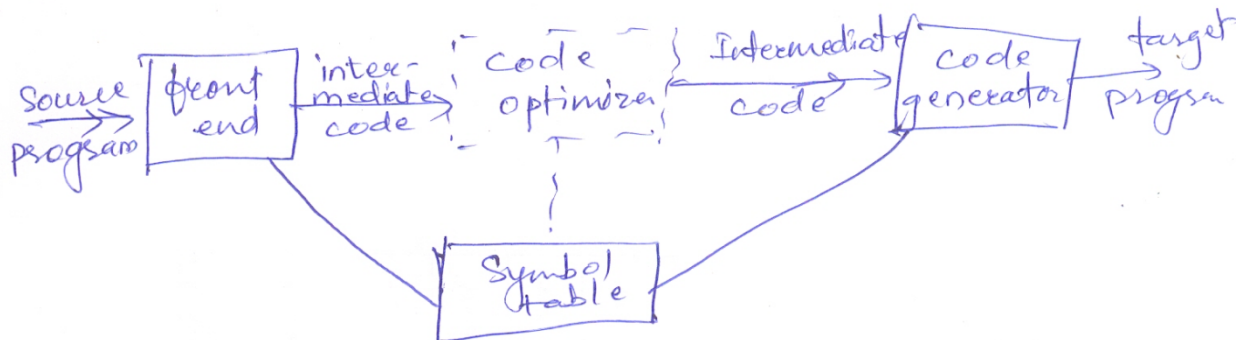


FIGURE Position of code generator

A Code-Generation algorithm-

The code generation algorithm takes a sequence of three-address statements as a input from a basic block. For each three-address statement of the form $x := y \text{ op } z$ we do the following actions:

1. To store the result of $y \text{ op } z$, determine a location L , by invoking **getreg** function. L may be a register or a memory location.
2. To determine y' , current location of y , consult address descriptor of y . If value of y is currently both in memory and register prefer register for y' . If value of y is not already in L then generate instruction **MOV L, y'** to place a copy of y' in L .
3. Generate instruction **op z', L** where z' is current location of z . If z is in both register and memory, prefer value from register. Update address descriptor of x to indicate value of x is in location L . If L is a register, update its register descriptor to indicate that it contains value of x and remove x from all other register descriptors.
4. If the current values of y and z are not used in future or they are not live after exit from the block and are in the register, alter the register descriptor to indicate that those register no longer contains the value of y and z .

The Function getreg

Function getreg returns the location for the value of x in the assignment $x = y \text{ op } z$.

1. If the name y is in a register that holds the value of no other names and y is not live and has no next use after execution of $x = y \text{ op } z$, then returns the register of y is no longer in L.
2. Failing 1, return an empty register for L if there is one.
3. failing 2, if x has a next use in a block, op is an operator, such as indexing, that requires a register, find an occupied register R. Store the value of R into a memory location (by MOV R,M) if it is not already in the proper memory location M, update the address descriptor for M, and return R. If R holds the value of several variables, a MOV instruction must be generated for each variable that needs to be stored. A suitable occupied register might be one whose value is also in memory.
4. If x is not used in the block, or no suitable occupied register can be found, select the memory location of x as L.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No.: C-54	Name: Quazi Md Moinuddin
Class: TE-C	Batch: C-3
Date of Experiment: 01.04.2025	Date of Submission: 15.04.2025
Grade:	

B.1 Software Code written by student:

(Paste your code completed during the 2 hours of practical in the lab here)

```
#include <stdio.h>
#include <string.h>
#include <conio.h>

struct instruction {
    char result[10];
    char arg1[10];
    char op;
    char arg2[10];
};

void main() {
    struct instruction code[10];
    int n, i, ch;
    clrscr();

    do {
        printf("Enter number of instructions: ");
        scanf("%d", &n);

        for (i = 0; i < n; i++) {
            printf("Enter instruction %d (format: a + b = t1):\n", i + 1);
            scanf("%s %c %s = %s", code[i].arg1, &code[i].op, code[i].arg2, code[i].result);
        }

        printf("\nGenerated Target Code:\n");

        for (i = 0; i < n; i++) {
            printf("MOV R0, %s\n", code[i].arg1);

            switch (code[i].op) {
                case '+': printf("ADD R0, %s\n", code[i].arg2); break;
            }
        }
    } while (ch != 'q');
```

```

        case '-': printf("SUB R0, %s\n", code[i].arg2); break;
        case '*': printf("MUL R0, %s\n", code[i].arg2); break;
        case '/': printf("DIV R0, %s\n", code[i].arg2); break;
        default : printf("Invalid operator\n"); break;
    }

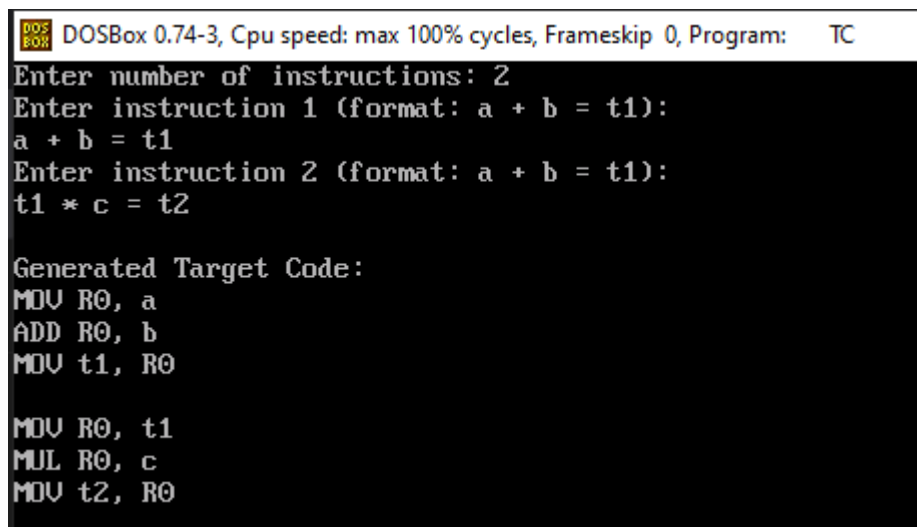
    printf("MOV %s, R0\n\n", code[i].result);
}

printf("Do you want to continue? (1 for Yes / 0 for No): ");
scanf("%d", &ch);
clrscr();
} while (ch == 1);

printf("Thank you for using the Target Code Generator!");
getch();
}

```

B.2 Input and Output:



```

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Enter number of instructions: 2
Enter instruction 1 (format: a + b = t1):
a + b = t1
Enter instruction 2 (format: a + b = t1):
t1 * c = t2

Generated Target Code:
MOV R0, a
ADD R0, b
MOV t1, R0

MOV R0, t1
MUL R0, c
MOV t2, R0

```

B.3 Observations and learning:

Understood how intermediate code bridges high-level source code and target machine code.

Learned how instructions are converted into low-level operations using registers.

Observed the importance of optimizing generated code for performance.

Gained hands-on experience with converting three-address code into assembly-like instructions.

B.4 Conclusion:

Target code generation is a crucial phase in the compiler design process. It translates intermediate code into executable machine-level instructions. Implementing this phase helps to understand how source-level logic is transformed into CPU-understandable format.

B.5 Question of Curiosity

1. *Differentiate Between Machine Dependent and Machine Independent code optimization.*

Machine Dependent Optimization	Machine Independent Optimization
Performed with specific hardware in mind	Performed without considering target machine
Utilizes specific instruction set or registers	General optimization regardless of architecture
Examples: register allocation, instruction scheduling	Examples: constant folding, dead code elimination
Affects performance on specific machine	Affects portability and general efficiency

2. *Describe Peephole Optimization.*

Peephole Optimization is a technique where a small window ("peephole") of instructions is examined and replaced with a more efficient sequence without changing the output.

- Example:

Before optimization:

```
MOV A, R0
MOV R0, A
```

After peephole optimization:

```
// Redundant MOV removed
```

- Common peephole optimizations:
 - Removing redundant instructions
 - Constant folding (evaluating constants at compile-time)
 - Strength reduction (e.g., replacing multiplication with addition)
 - Replacing sequences with simpler equivalents